香港教育大學
The Education University of Hong Kong

MSc(AIET) 2025-26

MTH6184 Data Mining in STEM Education

Chapter 8. Ensembles Methods

Instructor: Philip Yu

Chapter Outcomes

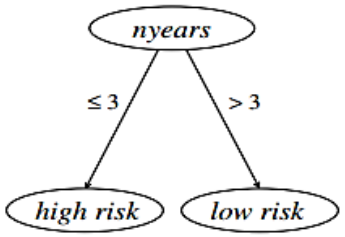
After completing this chapter, you are able to:

- Understand what ensemble methods are
- Perform bagging (bootstrap aggregating)
- Perform random forest
- Perform Adaboost and Gradient Tree Boosting
- Understand the power of boosting

Ensemble Methods

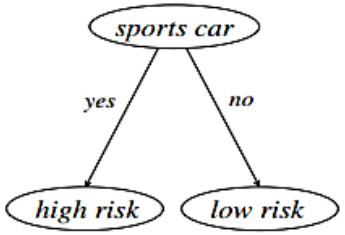
- Motivating example: A small change in a dataset can make the tree structure unstable.

nyears	sports car	risk
1	*yes*	*high*
2	*no*	*high*
2	*yes*	*high*
4	*no*	*low*
8	*no*	*low*



(a) original sample and the corresponding tree

nyears	sports car	risk
1	*yes*	*high*
4	*yes*	*high*
2	*yes*	*high*
4	*no*	*low*
8	*no*	*low*



(b) changed sample and the corresponding tree

Source: Li and Belford (2002). Instability of decision tree classification algorithms. SIGKDD'02.

- Ensemble methods aim to combine the predictions of several learners (not necessarily trees) in order to improve predictability / robustness (i.e., reduce variance) over a single learner. Two common types:
  - Bagging: reduce variance (e.g., Random forest)
  - Boosting: reduce bias (e.g. AdaBoost, Gradient tree boosting)

Improving STEM Performance by Leveraging Machine Learning Models

***Abstract*—This paper argues the importance of building an early warning system to improve the academic performance of post-secondary science, technology, engineering, and mathematics (STEM) education. It emphasizes the necessity of identifying not only the students who are at the risk of failing or dropping out of the course, but also the students who are not performing good (obtaining grade C) as well as who are doing good (grade A or B). For increasing the likelihood of graduation and improving retention, it is imperative to send early signals about predicted performance to the entire spectrum of the class. To accomplish this goal, this paper leverages Machine Learning based predictive models. More specifically, it develops multi-class classifiers to identify students belonging to three groups: Prone to risk (grade C or below), OK (grade B), and Good (grade A). Through an extensive analysis, it identifies the most effective classifiers for early forecasting of student performance.**

***Index Terms*—Predictive model, Machine Learning, Classifier, STEM Education, Early Warning**

Log Reg: Logistic Regression
KNN: K-Nearest Neighbor
Rand For: Random Forest
MLP: Multi-Layer Percptron
GBC: Gradient Boosting Classifier
ABC: Adaptive Boosting Classifier
NBC: Naïve Bayes Classifier

Model	Hyper-parameter values
Rand For	n-estimators = 1000, max_depth = 9
KNN	n-neighbors = 23, weights = uniform, p = 1
Log Reg	multi-class = multinomial, C = 0.007, max-iter = 500, solver = lbfgs

A. Dataset

In all experiments, the data are split into training and testing sets using 80% and 20% split, respectively. In other words, out of total 538 students, data obtained from 430 students is used in training the models and 108 for testing. The data are labelled

B. Features

Fourth prediction	Quiz 1 – 7 & Homework 1, 2, 3, 4 & Midterm 1	Week 1 – 12

PREDICTION 4: EVALUATION OF 7 MODELS

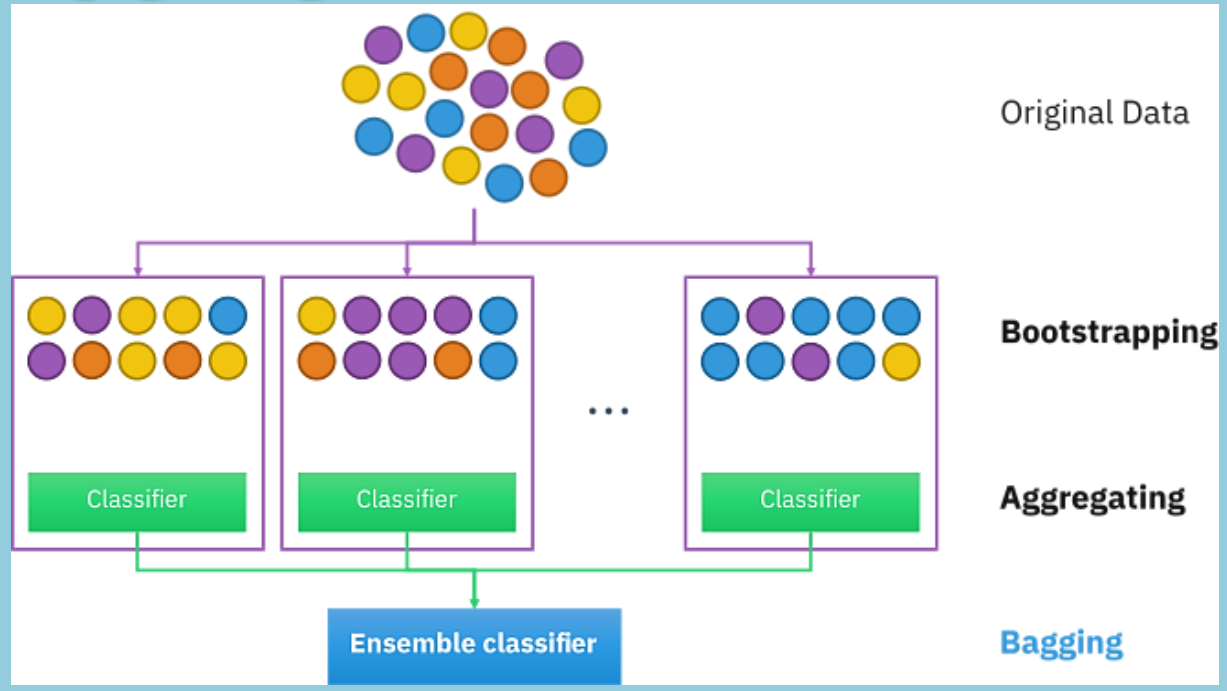
Method	Log Reg	KNN	Rand For	MLP	GBC	ABC	NBC
F1	0.832	0.825	0.798	0.767	0.724	0.744	0.775
Total Acc.	0.833	0.824	0.796	0.769	0.722	0.741	0.769
Total Recall	0.833	0.824	0.796	0.769	0.722	0.741	0.769
Recall Good	0.935	0.891	0.826	0.913	0.804	0.783	0.783
Recall OK	0.784	0.784	0.730	0.703	0.622	0.730	0.784
Recall PR	0.720	0.760	0.840	0.600	0.720	0.680	0.720

Recall PR: Recall (Prone-to-Risk)

$$\text{Recall (group x)} = \frac{\text{True Positive (group x)}}{\text{True Positive (group x) + False Negative (group x)}}$$

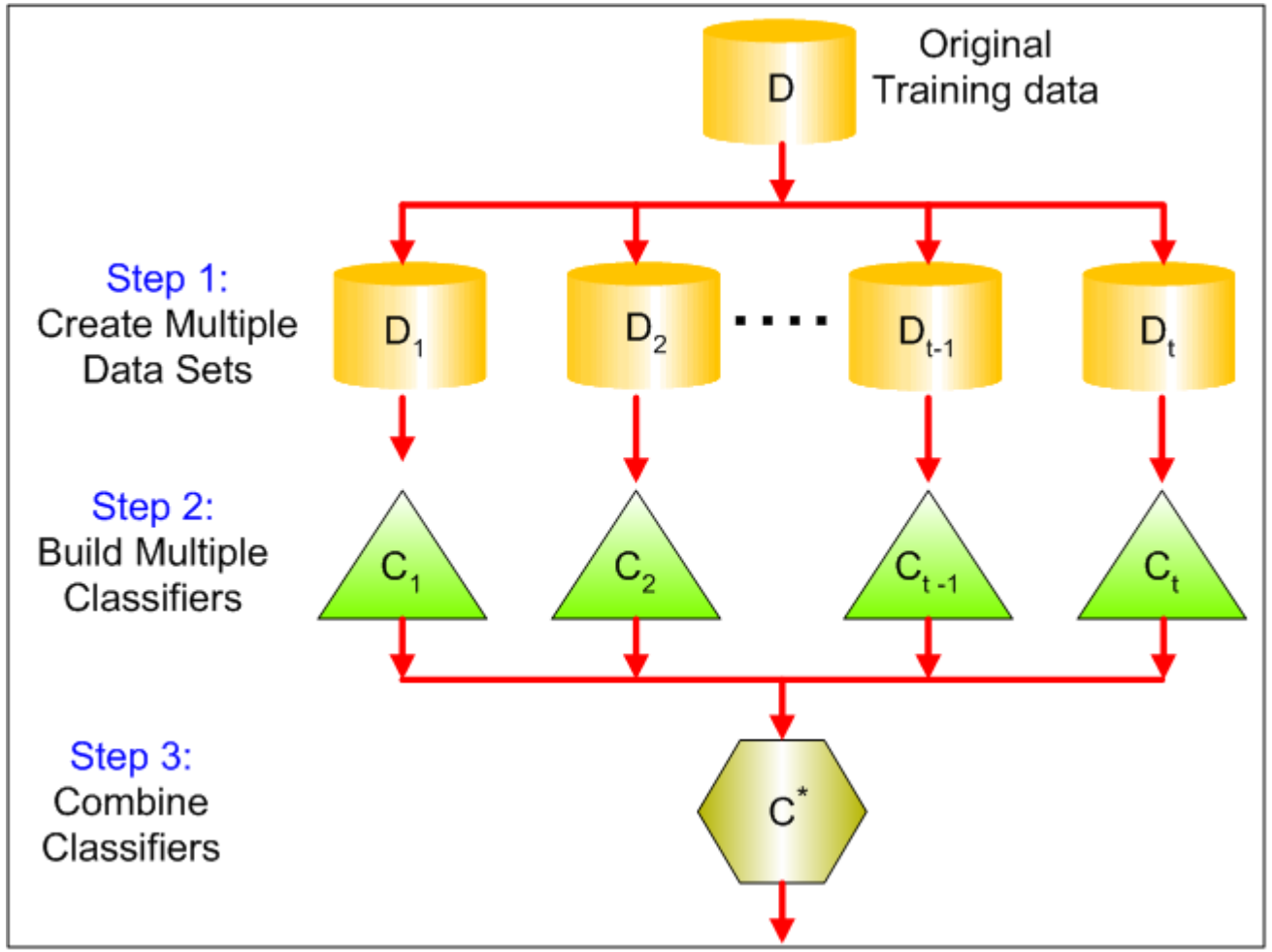
Aly, M. and Hasan, M.R. (2019). Improving STEM performance by leveraging machine learning models. In the Proceedings of the International Conference on Frontiers in Education: CS and CE, 205-211.

Bagging and Random Forest



Bagging

```
sklearn.ensemble.BaggingClassifier | BaggingRegressor:
base_estimator = DecisionTreeClassifier | DecisionTreeRegressor
bootstrap = True
n_estimators = 10
max_samples = 1.0 (100% of the samples) | integer (count)
max_features = 1.0 (100% of the features) | integer (count)
oob_score = False
```



Data sets can be dawn by bootstrapping (bagging) or random sampling without replacement (pasting)

To combine predictions, use averaging the predicted probability (or majority voting if the base-estimator does not generate probability) for classification and the average for regression

We can use out-of-bag (oob) samples to estimate the generalization error.

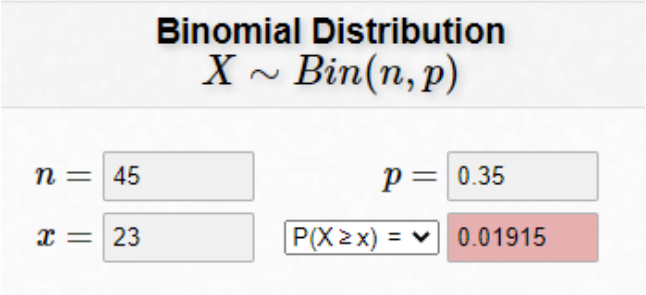
Why Bagging Works?

- Suppose there are $m = 25$ binary classifiers
  - Each classifier has an error rate, $\varepsilon = 0.35$
  - Assume the errors made by these classifiers are independent
  - Probability that the bagging classifier (majority voting) makes a wrong prediction (no. of mistakes, $X \geq \lceil m/2 \rceil = 13$):

$$P(X \geq 13) = \sum_{i=13}^{25} \binom{25}{i} \varepsilon^i (1-\varepsilon)^{25-i} = 0.0604$$

  - Ensembles usually improve when adding more classifiers. Eg. When $m = 45$, $P(X \geq \lceil m/2 \rceil) = 0.0192$

- How can we make independent errors for the 25 binary classifiers used in bagging?



https://homepage.divms.uiowa.edu/~mbognar/applets/bin.html

Bagging (Bootstrap aggregating (Breiman 1996))

- Bootstrapping: Sampling with replacement from training data
  - Each bootstrap sample has the same size as the original training data
  - Build a base classifier on each bootstrap sample
  - A test instance is assigned to the class with the highest number of votes or highest mean predicted probability

- Bagging Algorithm:

(1) Given a training set $\mathcal{D}$ of size n.

Let T be the number of bootstrap samples.

(2) For $t = 1$ to T, repeat steps (a)-(b):

(a) From $\mathcal{D}$, create a bootstrap sample of size n

(b) Train a classifier $C^{(t)}(\boldsymbol{x})$ based on the bootstrap sample

(3) Classify a test instance x by

Majority voting: $C(x) = \underset{k}{\operatorname{argmax}} \sum_{t=1}^{T} \mathbb{I}(C^{(t)}(\boldsymbol{x}) = k)$ Highest mean prob: $C(x) = \underset{k}{\operatorname{argmax}} \frac{1}{T} \sum_{t=1}^{T} P(C^{(t)}(\boldsymbol{x}) = k)$

Remarks on Bagging

- A bootstrap sample contains around 63.2% of the original data
  - Suppose the size of original training data is n
  - Each data point has probability $1-\left(1-\frac{1}{n}\right)^n$ of being selected in a bootstrap sample of size n
  - When $n \to \infty$, $1-\left(1-\frac{1}{n}\right)^n \to 1-\frac{1}{e} = 0.632$
  - On average, a bootstrap sample contains approximately 63.2% of the original data

- For continuous response,
  - use average prediction

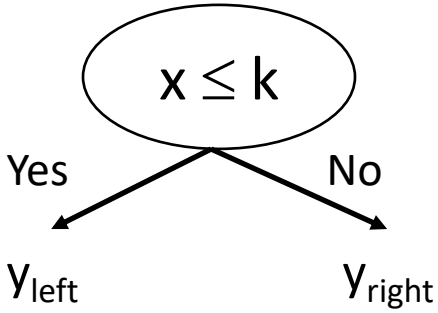
Bagging Example

- Consider 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump (tree with 1 split)
  - Decision rule: $\{x \leq k\}$ vs $\{x > k\}$
  - Split point k is chosen based on entropy reduction



Bagging Example (Cont.)

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

x <= 0.35 → y = 1
x > 0.35 → y = -1

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

x <= 0.7 → y = 1
x > 0.7 → y = 1

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.35 → y = 1
x > 0.35 → y = -1

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.3 → y = 1
x > 0.3 → y = -1

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

x <= 0.35 → y = 1
x > 0.35 → y = -1

Bagging Example (Cont.)

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 → y = -1
x > 0.75 → y = 1

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

x <= 0.75 → y = -1
x > 0.75 → y = 1

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 → y = -1
x > 0.75 → y = 1

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 → y = -1
x > 0.75 → y = 1

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

x <= 0.05 → y = 1
x > 0.05 → y = 1

- Summary of Training sets:

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

- Assume test set is the same as the original data
- Use majority vote to determine class of ensemble classifier

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Predicted Class Sign	1	1	1	-1	-1	-1	-1	1	1	1

Bagging Performance

- Testing Misclassification Rate (Percent)

UCI ML Dataset	err_{1tree}	err_{Bag}	Decrease
Waveform	29.0	19.4	33%
Heart	10.0	5.3	47%
Breast cancer	6.0	4.2	30%
Ionosphere	11.2	8.6	23%
Diabetes	23.4	8.8	20%
Glass	32.0	24.9	22%
Soybean	14.5	10.6	27%

50 bootstrap samples each

- Testing Mean Squared Error

UCI ML Dataset	err_{1tree}	err_{Bag}	Decrease
Boston	19.1	11.7	39%
Ozone	23.1	18.0	22%

50 bootstrap samples each

Class Activity 1:

Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. What would happen to Bagging?

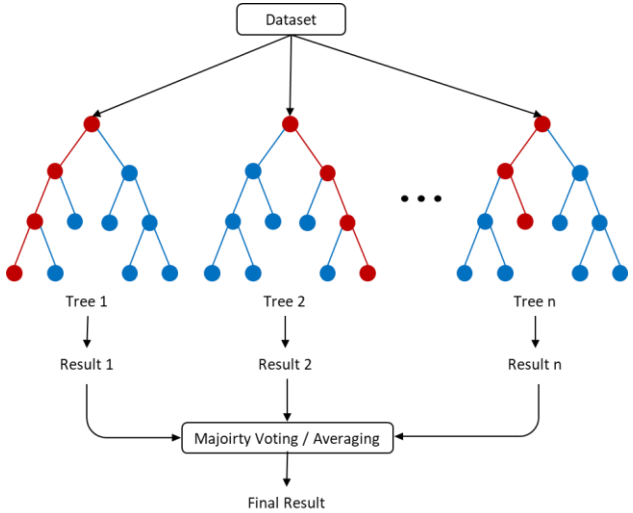
Random Forest

- In a random forest, each tree is built from a sample drawn with replacement (i.e., bootstrapping) from the training set.
- When splitting a node during the construction of a tree, a random subset of features is drawn (hence the term "Random"), from which the best split is selected.
- As a result of this randomness, the bias of the forest usually slightly increases (with respect to the bias of a single tree) but, due to averaging, its variance also decreases, usually more than compensating for the increase in bias, hence yielding an overall better model.
- In contrast to the original publication (Breiman,2001), the scikit-learn implementation combines classifiers by averaging their probabilistic prediction, instead of letting each classifier vote for a single class.
- A feature's importance can be measured by averaging the difference in score (eg. accuracy) before and after the permutation of the feature values of a fitted model (eg. random forest, logistic regression).

```
sklearn.ensemble.RandomForestClassifier:
n_estimators = 100
criterion = 'gini'|'entropy'
max_depth = None
max_features = 'sqrt'|'log2'|fraction|int.
bootstrap = True
oob_score = False
```

```
sklearn.ensemble.RandomForestRegressor:
n_estimators = 100
criterion = 'squared_error' | 'absolute_error'
max_depth = None
max_features = 1.0|'sqrt'|'log2'|fraction|int.
bootstrap = True
oob_score = False
```

```
sklearn.inspection.permutation_importance(
estimator, X, y,
scoring = 'accuracy' | 'r2' …,
n_repeats = 5)
```



Class Activity 2:

Random Forest Demo

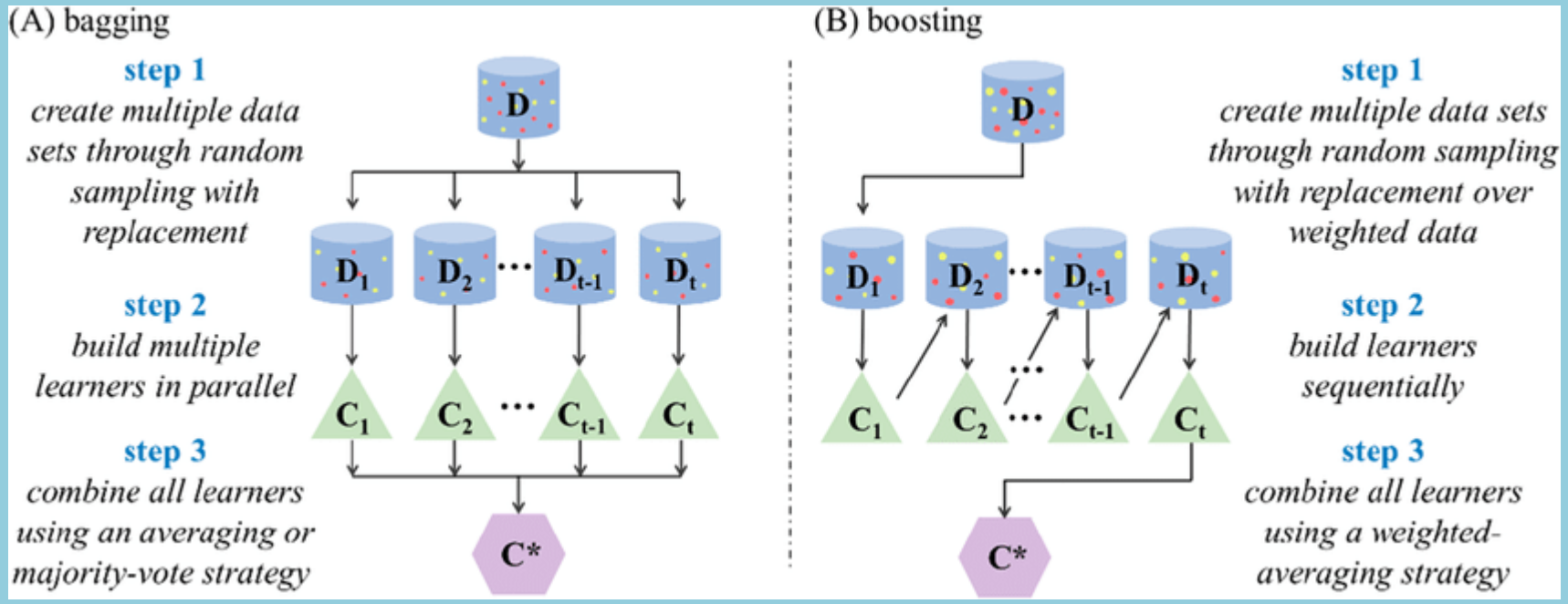
Pros and Cons of Bagging

- Pros
  - It can improve performance (eg. accuracy) for unstable* learning algorithms (eg. decision tree, neural network)
  - It can help reduce the variance, especially if the base classifiers/estimators are unstable, due to the averaging effect of majority voting. It does not, in general, have much effect on the bias.

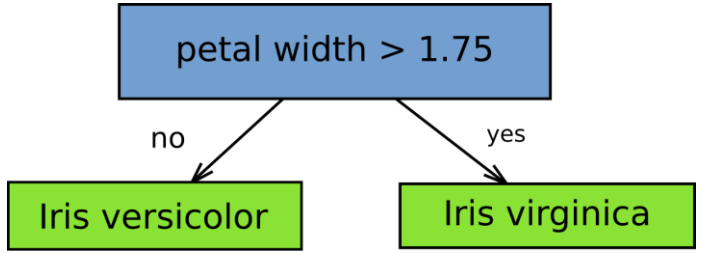
  * small changes in data lead to large changes in the prediction

- Cons
  - no simple, interpretable model
  - If the base classifier/estimator is stable, bagging may not improve the performance

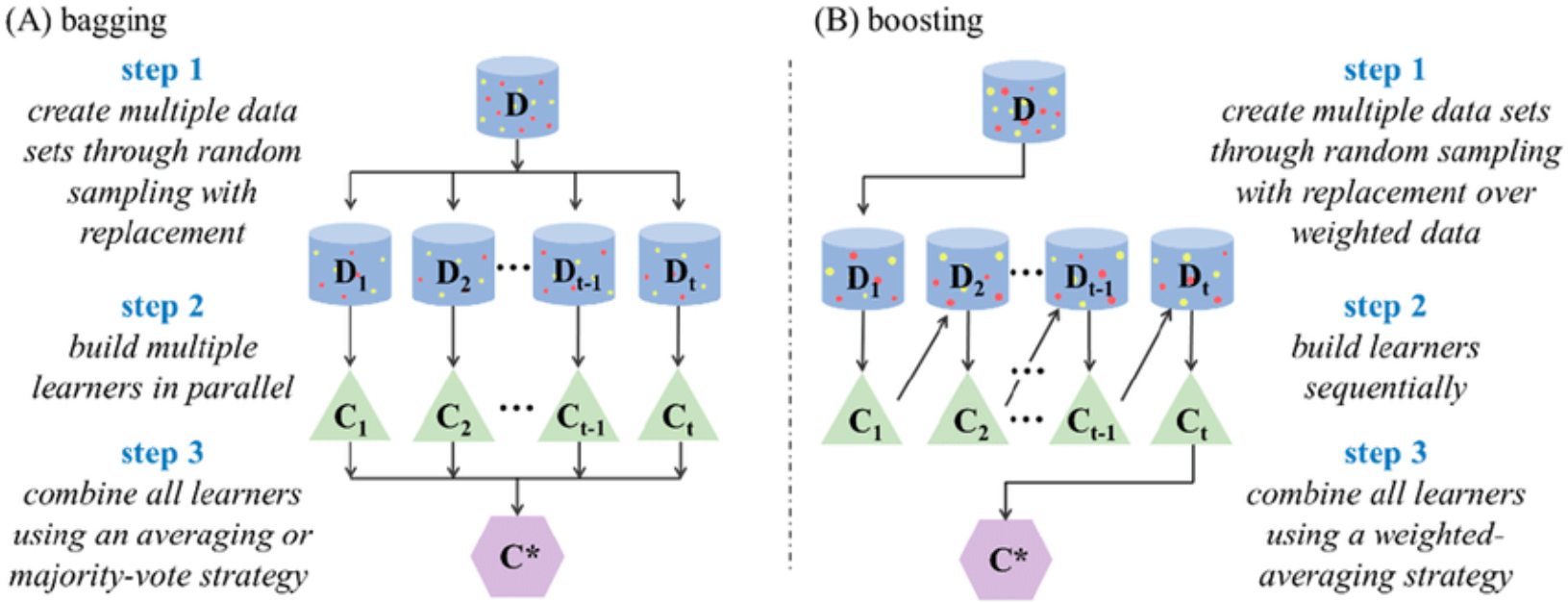
Boosting


(A) bagging
step 1
create multiple data sets through random sampling with replacement
step 2
build multiple learners in parallel
step 3
combine all learners using an averaging or majority-vote strategy
D
D1
D2
Dt-1
Dt
C1
C2
Ct-1
Ct
C*
(B) boosting
step 1
create multiple data sets through random sampling with replacement over weighted data
step 2
build learners sequentially
step 3
combine all learners using a weighted-averaging strategy
D
D1
D2
Dt-1
Dt
C1
C2
Ct-1
Ct
C*

Boosting

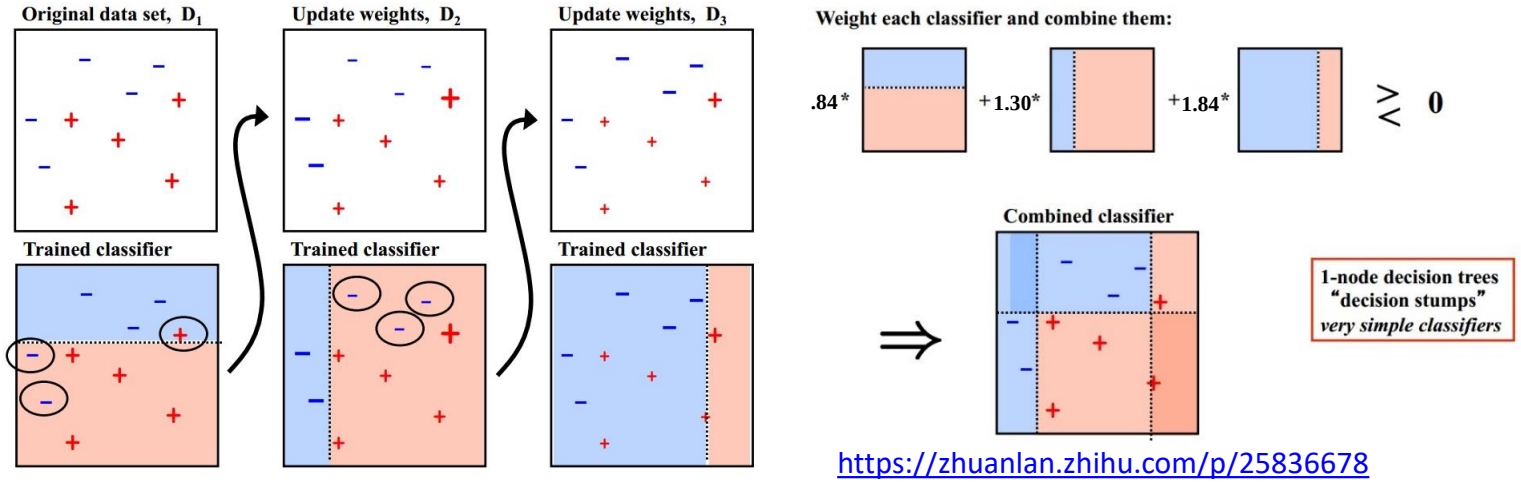


- Boosting fits weak learners (eg. decision stump) sequentially with information from previous fitted model
- In contrast, bagging or random forests that fit weak learners in parallel on each re-sampled dataset
- There are multiple boosting algorithms, including AdaBoost, Gradient Boosting, XGBoost (extreme gradient boosting), LightGBM, …



Boosting

- to *boost* the performance on previously misclassified observations
  - Initially, assign equal weights to all training observations
  - Unlike bagging, weights will be changed at the end of each boosting round
    - assign more weights to misclassified observations and
    - assign less weights to correctly classified observations



https://zhuanlan.zhihu.com/p/25836678

  - Each weak classifier (e.g. decision tree) in the sequence refocused on the observations that the previous ones found difficult and misclassified.
  - Unlike bagging which aims at variance reduction, boosting aims at bias reduction.
  - Two popular tree boosting models: Adaptive boosting (AdaBoost) and Gradient boosting.

Adaptive Boosting (AdaBoost) Algorithm

- First developed by Freund and Schapire (1997) for binary classification and then extended to multi-class classification by Zhu, et al (2009).

SAMME Algorithm by Zhu, eta al. (2009)

- Given a training set $\{\boldsymbol{x}_i, y_i\}_{i=1}^n$ in a multi-class classification of K classes, i.e., $y_i \in \{1,2,\dots,K\}$.
- Initialize the training observation weights $w_i = \frac{1}{n}, i = 1,\dots,n$
- For $t = 1,\dots,T$, repeat steps (a)-(e):

  (a) Fit a weak classifier $C^{(t)}(\boldsymbol{x})$ (tree, says) to the training data with weights $w_i, i = 1,\dots,n$

  (b) Apply $C^{(t)}(\boldsymbol{x})$ to the original data to compute weighted error rate: $err^{(t)} = \sum_{i=1}^n w_i \mathbb{I}(C^{(t)}(\boldsymbol{x}_i) \neq y_i)$

  (c) If error rate $err^{(t)} > 1 - \frac{1}{K}$, set $w_i = \frac{1}{n}$ and go back to step (a)

  (d) Compute classifier's importance: $\alpha^{(t)} = \eta\left[\ln\left(\frac{1-err^{(t)}}{err^{(t)}}\right) + \ln(K-1)\right] \geq 0, 0 < \eta \leq 1$ (learning rate)

  (e) Update weights: $w_i \leftarrow w_i \exp\left[\alpha^{(t)} \cdot \mathbb{I}(C^{(t)}(\boldsymbol{x}_i) \neq y_i)\right], i = 1,\dots,n$ and renormalize w_i to sum to 1
- After training the T classifiers, the combined classifier is:

$$C(\boldsymbol{x}) = \underset{k}{\operatorname{argmax}} \sum_{t=1}^{T} \alpha^{(t)} \cdot \mathbb{I}(C^{(t)}(\boldsymbol{x}) = k)$$

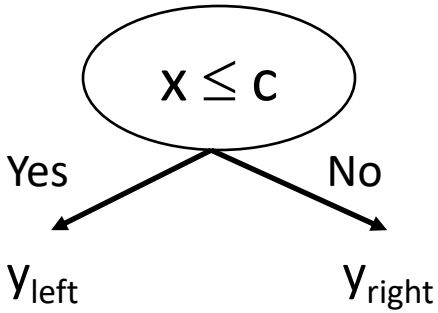
AdaBoost Example

- Consider 1-dimensional data set with $K = 2$:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Based classifier is a decision stump
  - Decision rule: $\{x \leq c\}$ vs $\{x > c\}$
  - Split point k is chosen based on entropy reduction (i.e. information gain)



AdaBoost Example

- Simulated training sets for the first 3 boosting rounds:

Boosting Round 1:

x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Boosting Round 2:

x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Boosting Round 3:

x	0.2	0.2	0.4	0.4	0.4	0.4	0.5	0.6	0.6	0.7
y	1	1	-1	-1	-1	-1	-1	-1	-1	-1

- Summary: $(\eta = 1)$

Apply the fitted tree to original data to compute weighted error

Round	Split point	Left class	Right class	Weighted Error	alpha
1	0.75	-1	1	30%	0.8473
2	0.05	1	1	28.57%	0.9163
3	0.3	1	-1	15%	1.7346

AdaBoost Example

- Weights

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.167	0.167	0.167	0.071	0.071	0.071	0.071	0.071	0.071	0.071
3	0.117	0.117	0.117	0.125	0.125	0.125	0.125	0.05	0.05	0.05

- Classification

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	1.80	1.80	1.80	-1.67	-1.67	-1.67	-1.67	0.029	0.029	0.029
Sign	1	1	1	-1	-1	-1	-1	1	1	1

Predicted Class

Tree Boosting Remarks

```
sklearn.ensemble.AdaBoostClassifier:
base_estimator = DecisionTreeClassifier(max depth=1)
n_estimators = 50
learning_rate = 1
algorithm = 'SAMME.R' | 'SAMME'
```

- Since AdaBoost can boost the performance of weak classifiers such as decision trees, the most popular weak decision trees would be decision stumps, decision trees with depth 1.

- Across the stages, Tree 1, Tree 2, …, Tree T are generated and they are dependent.

- SAMME vs SAMME.R:

SAMME.R
Zhu, eta al. (2009)

- Initialize the training observation weights $w_i = \frac{1}{n}, i = 1, \dots, n$
- For $t = 1, \dots, T$, repeat steps (a)-(e):
  (a) Fit a classifier $C^{(t)}(\boldsymbol{x})$ to training data using weights $w_i, i = 1, \dots, n$
  (b) Obtain the class probability estimates $p_y^{(t)}(\boldsymbol{x}) = P_w(Y = k|\boldsymbol{x}), k = 1, \dots, K$
  (c) Set $f_k^{(t)}(\boldsymbol{x}) = (K-1)\left(\ln p_k^{(t)}(\boldsymbol{x}) - \frac{1}{K}\sum_{c=1}^{K} \ln p_c^{(t)}(\boldsymbol{x})\right)$
  (d) Update obs. weights: $w_i \leftarrow w_i \exp\left[-\eta \frac{K-1}{K} \boldsymbol{y}_i^{\mathsf{T}} \ln \boldsymbol{p}^{(t)}(\boldsymbol{x}_i)\right], i = 1, \dots, n$ and renormalize w_i to sum to 1
- After training the T classifiers, the combined classifier is: $C(\boldsymbol{x}) = \underset{k}{\operatorname{argmax}} \sum_{t=1}^{T} f_k^{(t)}(\boldsymbol{x})$

$$\boldsymbol{y} = (y_1, \dots, y_K)^{\mathsf{T}} \text{ with } y_k = \begin{cases} 1 & \text{if } Y = k \\ -\frac{1}{K-1} & \text{if } Y \neq k \end{cases} \qquad \boldsymbol{p}^{(t)}(\boldsymbol{x}) = \left(p_1^{(t)}(\boldsymbol{x}), \dots, p_K^{(t)}(\boldsymbol{x})\right)^{\mathsf{T}}$$

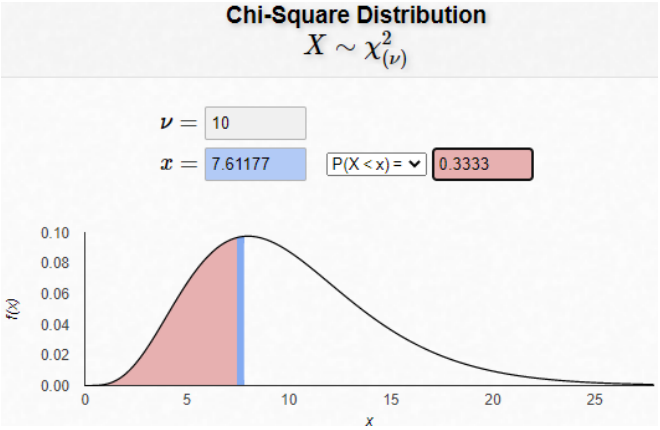
Simulation: Boosting with Decision Trees

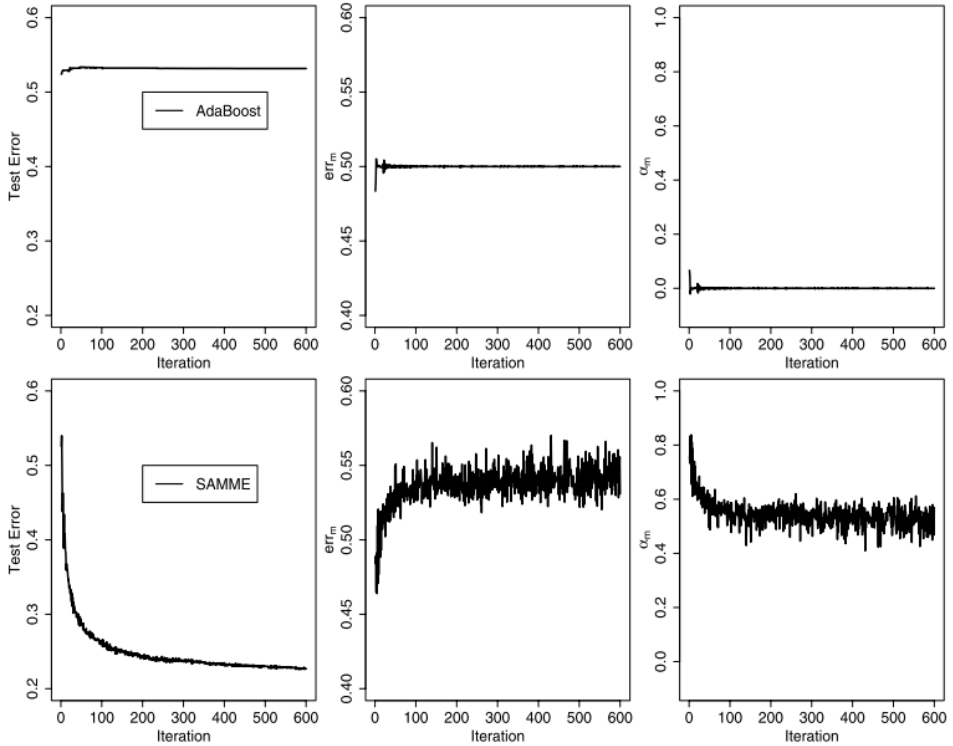
- Training: 3000 obs. simulated from

$$Y = \begin{cases} 1 & \text{if } 0 \le \Sigma_{j=1}^{10} x_j^2 < \chi^2_{10,1/3} = 7.61 \\ 2 & \text{if } \chi^2_{10,1/3} \le \Sigma_{j=1}^{10} x_j^2 < \chi^2_{10,2/3} = 11.32 \\ 3 & \text{if } \chi^2_{10,2/3} \le \Sigma_{j=1}^{10} x_j^2 \end{cases}$$

  where the x_j's are iid N(0, 1)

- Testing: 10000 obs.
- Ten-terminal node trees are used as weak classifiers.



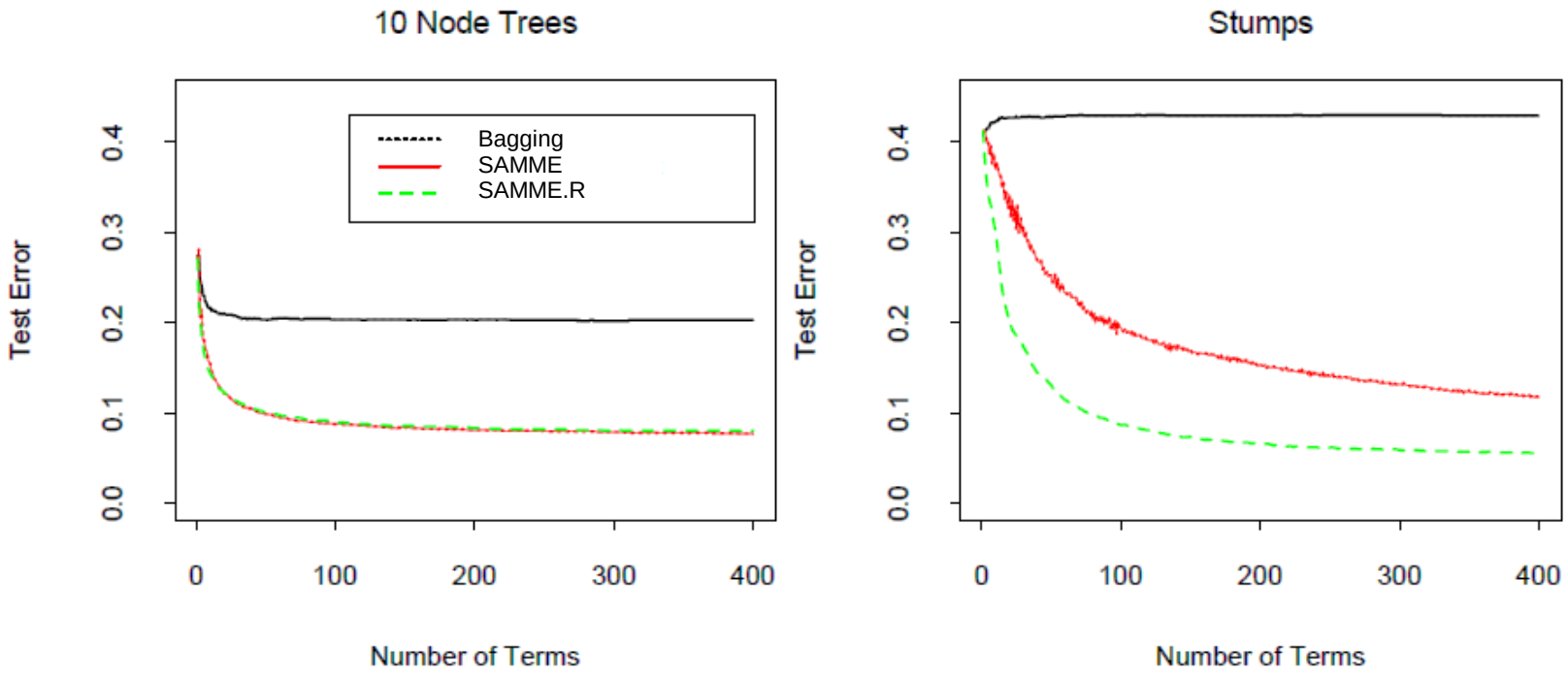


Simulation: Bagging, SAMME and SAMME.R

- Training: 2000 obs. simulated from

$$Y = \begin{cases} 1 & \text{if } \Sigma_{j=1}^{10} x_j^2 > \chi^2_{10,\frac{1}{2}} = 9.34 \\ -1 & \text{otherwise} \end{cases}$$

where the x_j's are iid N(0, 1)



Why Boosting Works?

- Boosting builds an additive model for a classification / regression problem:

$$\boldsymbol{f}^{(T)}(x) = \sum_{t=1}^{T} \beta_t b_t(x;\gamma_t) = \boldsymbol{f}^{(T-1)}(x) + \beta_T b_T(x;\gamma_T)$$

  where $b_t(x;\gamma_t)$ is a model with parameters γ_t (eg. a tree with the splits depending on γ_t)
- Many of the learning techniques are of this form. Eg. Linear regression
- Traditionally the parameters (β_t, γ_t) are fit jointly (e.g. LSE, MLE)
- With boosting, the parameters (β_t, γ_t) are fit in a stagewise fashion. This slows the process down, and tends to overfit less quickly.
- SAMME and SAMME.R are equivalent to forward Stagewise Additive Modeling using a Multi-class Exponential loss function:

$$L_{SAMME} = \sum_{i=1}^{n} \exp\left[-\frac{1}{K}\boldsymbol{y}_i^{\intercal}\boldsymbol{f}^{(T)}(\boldsymbol{x}_i)\right] \qquad L_{SAMME.R} = E\left(\exp\left[-\frac{1}{K}\boldsymbol{y}^{\intercal}\boldsymbol{f}^{(T)}(\boldsymbol{x})\right]\right)$$

- Note: Binary logistic regression: $y \in \{1, -1\}$, let $f(\boldsymbol{x}) = \boldsymbol{x}^{\intercal}\boldsymbol{\beta}$

$$P(y=1|x) = \frac{1}{1+\exp(-f(\boldsymbol{x}))} \qquad P(y=-1|x) = \frac{\exp(-f(\boldsymbol{x}))}{1+\exp(-f(\boldsymbol{x}))} = \frac{1}{1+\exp(f(\boldsymbol{x}))}$$

- Maximizing the log-likelihood is equivalent to minimizing

$$\text{Loss} = \sum_{i=1}^{N} \ln(1+\exp(-y_i f(x_i)))$$

Gradient Tree Boosting

- Recall: Forward stagewise additive modeling aims to minimize

$$Loss(f) = \sum_{i=1}^{N} L(y_i, f(x_i)) \text{ where } f(x) = \sum_{t=1}^{T} \beta_t b_t(x; \gamma_t)$$

- But in gradient boosting, $b_t(x; \gamma_t)$ is determined using gradient.
- Using gradient descent method to minimize $Loss(f)$, at iteration t,

$$\boldsymbol{f}_t = \boldsymbol{f}_{t-1} - \alpha_t \boldsymbol{g}_t \quad \boldsymbol{f}_t = (f_t(x_1), \dots, f_t(x_N))'$$

where gradient $\boldsymbol{g}_t = (g_{1t}, \dots, g_{Nt})'$ with $g_{it} = \left.\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right|_{f(x_i)=f_{t-1}(x_i)}$ and α_t is determined by minimizing $Loss(f_{t-1} - \gamma_t g_t)$.

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$\lvert y_i - f(x_i)\rvert$	$\text{sign}[y_i - f(x_i)]$

- However, g_{it}'s are evaluated based on training data only, it is not available for testing data. A possible solution is to approximate it by a tree $b_t(x_i; \theta)$: $\tilde{\theta}_t = \arg\min \sum_{i=1}^{N} (-g_{it} - b_t(x_i; \theta))^2$
- Note `xgboost` (Python package, YouTube) is much faster than `sklearn.ensemble.GradientBoostingClassifier`.

Which is better: Bagging or Boosting?

- No “best” method for all problems
- Computational considerations
- Interpretable models
- Some large-scale experiments revealed that
  - Boosting has a lower testing error
  - Although Boosting algorithms have been crowned as the “most accurate available off-the-shelf classifiers on a wide variety of data sets”, they are sensitive to noise and outliers, especially for small data sets.
  - Variants of bagging, for example, random forests, have been found to be comparable to AdaBoost.

Summary

Bagging	• Reduces variance and increases accuracy • Robust against outliers or noisy data • Often used with decision trees (i.e., random forest)
Boosting	• Also reduces variance and increases accuracy • Not robust against outliers or noisy data • Flexible — can be used with any loss function

Class Activity 3:

Lab 7: Calories Burnt Prediction

Thank you